

# CS541 Project 5: Reflections

## Synopsis

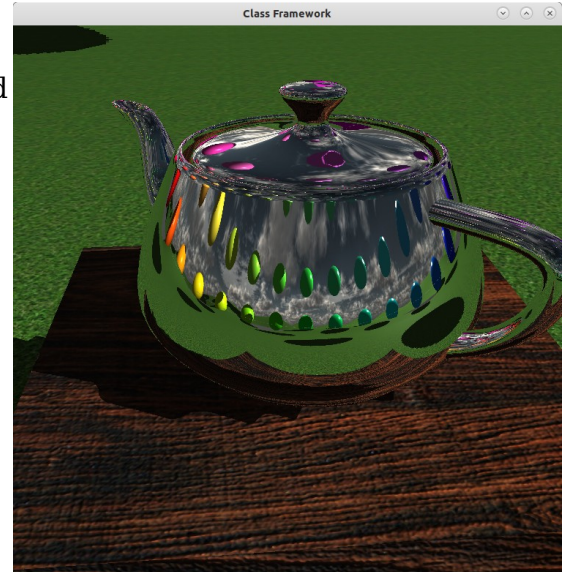
Implement a reflection-map algorithm using a dual paraboloid or equivalent. Then use the reflection map to render a reflective object in the scene.

## Instructions

Implement a reflection map to reflect the (dynamically changing) environment in a reflective object. In the standard framework for this class, use the central teapot to reflect the remaining objects in the scene.

The reflective object should reflect all the graphical features implemented so far, in particular lighting, shadows, and any other features implemented.

**Note:** The walls/room in the original scene do not play well with the reflection map. They should be disabled for this project. By commenting out the line `objectRoot→add(room, ...)`



## Details

This project will consist of the following passes:

**Shadow map pass:** No changes need to be made to the shadow-map pass, but this pass must be done first so that both the reflection passes and the final lighting pass can use the result to implement shadows.

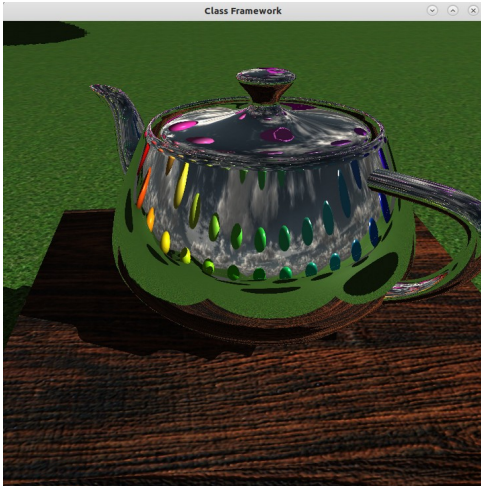
**Reflection passes:** These two passes are normal full lighting passes that differ only slightly from the lighting pass which was implemented in the previous project. The differences are:

- The teapot should not be drawn in this pass.
- The reflection pass is performed twice, one for the top half (which is drawn into one FBO) and again for the bottom half (which is drawn into another FBO).
- The view and perspective transformations usually used in the vertex shader are replaced with the paraboloid map. (See below.) This affects only the `gl_Position` calculation. The `ModelTransformation` is still used to map vertices into world coordinates.
- The “EYE” position for the reflection calculations is now the center-of-reflection, i.e., the center of the teapot. For this framework, the teapot is centered at about  $(0, 0, 1.5)$ .
- None of the lighting calculations in either the vertex or pixel shaders are changed.

**Final lighting pass:** This is another full lighting pass (the third one in this project) that also differs only slightly from the previous project and the two reflection passes. The differences are:

- The teapot should be drawn in this pass, but with a uniform boolean indicating it is the reflective object. All other objects should be drawn with that boolean indicating a non-reflective object.
- Being the lighting pass it is run only once, drawing to the screen (so no FBO), and using the usual view and perspective transformations. The EYE in this pass is the usual camera position.
- The normal lighting calculation is performed, but after that, if the boolean indicates that the reflective object is being drawn, a further reflective color is looked up from one of the reflection FBOs (see below) and combined with the lighting calculation results according to one of the three choices shown below.

## Combining lighting calculation and reflection calculation.



*Output the reflection color directly, completely ignoring the calculated lighting color. This results in an unnaturally perfect reflective object.*



*Output a simple combination of the lighting calculation and the reflection calculation. For instance*

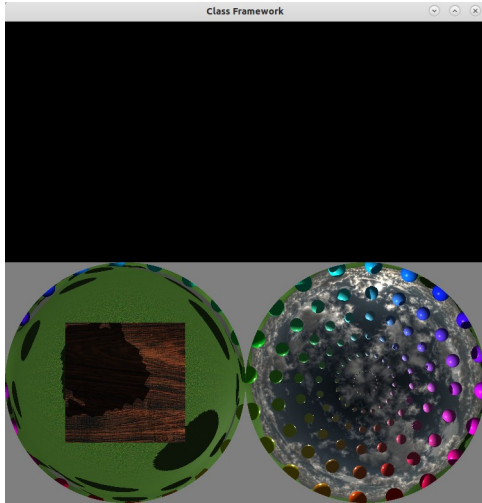
$$\text{lighting} + 0.1 * \text{reflection}$$



Consider the reflected light as a specular reflection and use the specular portion of the BRDF to combine this new source of light:  
That is

$$\text{lighting} + I(N \cdot L)BRDF_s(L, V)$$

where I is now the reflected color, and L is now the direction from which it comes (R in the notation below).



*The upper and lower reflection maps displayed side-by-side.*

## Map of the sphere of unit directions

When creating a reflection map, one must decide on a map from the **set of all 3D directions** to the **set of uv points** in one or more rectangular textures. Many such maps exist, see the Wikipedia page [https://en.wikipedia.org/wiki/List\\_of\\_map\\_projections](https://en.wikipedia.org/wiki/List_of_map_projections) for a list of nearly 70 used in map-making. We will use the **dual paraboloid** map, common in many Computer Graphics applications.

*Note: When the graphics pipeline renders objects to the render target, the clipper is still running, so the Z-coordinates in each of the following are carefully manipulated to fall within the clippers  $[-1, 1]$  range. (See the equations  $\pm c \|R\|/1000 - 1.0$ )*

### Dual Paraboloid map:

- **Two reflection passes, one each for the upper (c=+1) and lower(c=-1) FBOs.**
  - **Application**  
Draw all objects **except** the reflective object.
  - **Vertex shader:**  
Set the eye position EYE = center-of-reflection  
Calculate the world point P = ModelTR\*vertex  
Calculate the output by using the dual-paraboloid map instead of V and P.  
 $R = P - EYE$   
 $(a, b, c) = R/\|R\|$   
 $gl\_Position = (a/(1 \pm c), b/(1 \pm c), \pm c\|R\|/1000 - 1.0, 1.0)$   
Set up all normal lighting calculations (remembering EYE=center-of-reflection)
  - **Pixel shader:**  
All normal lighting calculations, including shadows, BRDF, textures, ...
- **Lighting pass**
  - **Application**  
Draw all objects with a boolean specifying **reflective** (teapot) or **not** (everything else)
  - **Vertex shader:**  
Set up all normal lighting calculations (with EYE as the usual camera position)
  - **Pixel shader:**  
All normal lighting calculations, including shadows, BRDF, textures, ...  
If the object being drawn is the marked by the **reflective** boolean,, then:  
 $R = 2(V \cdot N) N - V$  // View direction reflected across the normal  
 $(a, b, c) = R/\|R\|$   
If  $c > 0$  access the upper reflective texture using the  $+c$  branch) for uv  
otherwise access the lower reflective texture using the  $-c$  branch for uv  
where  $uv = (a/(1 \pm c), b/(1 \pm c)) * \frac{1}{2} + (\frac{1}{2}, \frac{1}{2})$   
  
Combine the reflective color with the lighting color as suggested above.

## GLSL hints

The reflection map algorithm uses the full lighting calculation (BRDF, shadows, textures, ...) three times - two reflection shaders, and the final lighting shader. In order to avoid duplicating code across all three instances, you should move the lighting calculation into a separate procedure in its own file. GLSL does not implement **#include** to facilitate this, but it can still be achieved. Here is an outline of the relevant parts.

### Final Vertex Shader:

```
void LightingVertex(vec3 eye); // Signature of the separated procedure
void main()
{
    gl_Position = ...
    eye = ...
    LightingVertex(eye);
}
```

### Final Fragment Shader:

```
vec3 LightingPixel(); // Signature of the separated procedure
void main()
{
    gl_FragColor.xyz = LightingPixel();
}
```

### Separated out lighting vertex shader:

```
// All the uniform, in, and out variables in the original vertex shader
void LightingVertex(vec3 eye)
{
    // All the code from the original vertex shader
}
```

### Separated out lighting pixel shader:

```
// All the uniform, in, and out variables in the original pixel shader
vec3 LightingPixel()
{
    // All the code from the original vertex shader
}
```

### The application can build the shaders from multiple files this way:

```
lightingProgram = new ShaderProgram();
lightingProgram->AddShader("final.vert", GL_VERTEX_SHADER);
lightingProgram->AddShader("final.frag", GL_FRAGMENT_SHADER);
lightingProgram->AddShader("lighting.vert", GL_VERTEX_SHADER); //NEW
lightingProgram->AddShader("lighting.frag", GL_FRAGMENT_SHADER); //NEW

glBindAttribLocation(lightingProgram->programId, 0, "vertex");
glBindAttribLocation(lightingProgram->programId, 1, "vertexNormal");
glBindAttribLocation(lightingProgram->programId, 2, "vertexTexture");
glBindAttribLocation(lightingProgram->programId, 3, "vertexTangent");
lightingProgram->LinkProgram();
```